Saito

SMART CONTRACT AUDIT

ZOKYO.

Apr 7, 2021 | v. 1.0

PASS

Zokyo's Security Team has concluded that this smart contract passes security qualifications to be listed on digital asset exchanges.

TECHNICAL SUMMARY

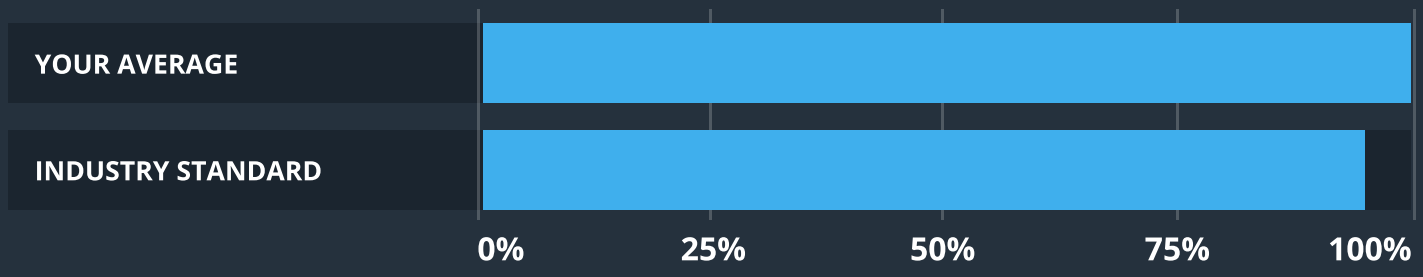
This document outlines the overall security of the Saito smart contracts, evaluated by Zokyo's Blockchain Security team.

The scope of this audit was to analyze and document the Saito smart contract codebase for quality, security, and correctness.

Contract Status

There were no critical issues found during the audit.

Testable Code



The testable code is 100%, which is above the industry standard of 95%.

It should be noted that this audit is not an endorsement of the reliability or effectiveness of the contract, rather limited to an assessment of the logic and implementation. In order to ensure a secure contract that's able to withstand the Ethereum network's fast-paced and rapidly changing environment, we at Zokyo recommend that the Saito team put in place a bug bounty program to encourage further and active analysis of the smart contract.

TABLE OF CONTENTS

Auditing Strategy and Techniques Applied 3
Executive Summary 4
Structure and Organization of Document 5
Manual Review 6
Code Coverage and Test Results for all files 8
Tests written by Saito team 8
Tests written by Zokyo Secured team 9

AUDITING STRATEGY AND TECHNIQUES APPLIED

The Smart contract's source code was taken from the Saito repository – https://github.com/SaitoTech/saito-evm-token/.
Last commit – a586e19a6d3152bc6c4bd3b2985db40fd1d63d42.

Requirements:

LitePaper – https://saito.io/saito-litepaper.pdf
WhitePaper – https://saito.io/saito-whitepaper.pdf

Throughout the review process, care was taken to ensure that the token contract:

- Implements and adheres to existing Token standards appropriately and effectively;
- Documentation and code comments match logic and behavior;
- Distributes tokens in a manner that matches calculations;
- Follows best practices in efficient use of gas, without unnecessary waste;
- Uses methods safe from reentrance attacks;
- Is not affected by the latest vulnerabilities;
- Whether the code meets best practices in code readability, etc.

Zokyo's Security Team has followed best practices and industry-standard techniques to verify the implementation of Bondly smart contracts. To do so, the code is reviewed line-by-line by our smart contract developers, documenting any issues as they are discovered. Part of this work includes writing a unit test suite using the Truffle testing framework. In summary, our strategies consist largely of manual collaboration between multiple team members at each stage of the review:

1	Due diligence in assessing the overall code quality of the codebase.	3	Testing contract logic against common and uncommon attack vectors.
2	Cross-comparison with other, similar smart contracts by industry leaders.	4	Thorough, manual review of the codebase, line-by-line.

EXECUTIVE SUMMARY

There were no critical issues found during the audit. All the mentioned findings may have an effect only in case of specific conditions performed by the contract owner.

There were three issues found during the audit. They are insignificant and have no major impact on the security level of the contract. All of the aforementioned issues were marked as “Informational” as they have no impact on the contract’s ability to operate. Nevertheless, Zokyo team recommends investigating those issues in order to enhance the contract performance.

Overall, the contracts are well written and structured. The findings during the audit have no impact on contract performance or security, so it is fully production-ready.

STRUCTURE AND ORGANIZATION OF DOCUMENT

For ease of navigation, sections are arranged from most critical to least critical. Issues are tagged “Resolved” or “Unresolved” depending on whether they have been fixed or addressed. Furthermore, the severity of each issue is written as assessed by the risk of exploitation or other unexpected or otherwise unsafe behavior:

Critical

The issue affects the ability of the contract to compile or operate in a significant way.

High

The issue affects the ability of the contract to compile or operate in a significant way.

Medium

The issue affects the ability of the contract to operate in a way that doesn’t significantly hinder its behavior.

Low

The issue has minimal impact on the contract’s ability to operate.

Informational

The issue has no impact on the contract’s ability to operate.

MANUAL REVIEW

No SPDX-License-Identifier found

INFORMATIONAL | RESOLVED

Trust in smart contract can be better established if their source code is available. Since making source code available always touches on legal problems with regards to copyright, the Solidity compiler encourages the use of machine-readable SPDX license identifiers.

Recommendation:
Every source file should start with a comment indicating its license.

```
// SPDX-License-Identifier: UNLICENSED
```

Unnecessary read function

INFORMATIONAL | RESOLVED

Public variables can be accessed directly, without additional function. Contract will become less in size and cheaper to deploy.

Recommendation:
Unless there is a special requirement, variable mintingNonce can be set to public and function getMintingNonce can be removed. But, if another contract will interact with this variable, it would be ok to keep getMintingNonce function, since this kind of interaction will be a little bit more gas efficient.

Unnecessary virtual and public keywords

INFORMATIONAL | RESOLVED

If the current contract isn't supposed to be inherited, no need to add a “virtual” keyword for incrementNonce(), burn(), mint() functions.

Also, those methods don’t seem to be called internally, so public keywords can be replaced with External.

Recommendation:
Unless there is a special requirement, keywords “public virtual” can be replaced with “external” on lines 24, 43, 62.

CODE COVERAGE AND TEST RESULTS FOR ALL FILES

Tests written by Saito team

```
Contract: Test Contracts
    SaitoToken Tests
      ✓ saitoToken decimals is 18 (73ms)
      ✓ saitoToken name is set (66ms)
      ✓ saitoToken ticker is set (62ms)
      ✓ saitoToken initial supply is 0 (57ms)
      ✓ saitoToken cannot be minted with a bad sig (381ms)
      ✓ saitoToken can be minted (462ms)
      ✓ saitoToken minting cannot be replayed (180ms)
      ✓ saitoToken can be minted by signing the next nonce (479ms)
      ✓ saitoToken can be transferred (294ms)
      ✓ saitoToken can be transferred through allowance (545ms)
      ✓ saitoToken cannot transfer more than allowance allowance()/transferFrom() (256ms)
      ✓ saitoToken burn publishes data as event (261ms)
      ✓ saitoToken can burn (282ms)
      ✓ saitoToken owners can increment nonce (243ms)

  14 passing (4s)
```

FILE	% STMTS	% BRANCH	% FUNCS	% LINES	UNCOVERED LINES
contracts\	100.00	66.67	100.00	100.00	
SaitoToken.sol	100.00	66.67	100.00	100.00	
All files	**100.00**	**66.67**	**100.00**	**100.00**	

Tests written by Zokyo Secured team

As part of our work assisting Saito in verifying the correctness of their contract code, our team was responsible for writing integration tests using the Truffle testing framework.

Tests were based on the functionality of the code, as well as a review of the Saito contract requirements for details about issuance amounts and how the system handles these.

```
Contract: SaitoToken
  ✓ should deploy with correct name (131ms)
  ✓ should deploy with correct symbol (99ms)
  ✓ should deploy with expected decimals (75ms)
  ✓ should deploy with expected initial total supply (83ms)
  ✓ should initial minting nonce be correct (91ms)
  ✓ should mint saito tokens correct (741ms)
  ✓ shouldn't mint saito tokens by not an owner (319ms)
  ✓ shouldn't mint saito tokens if signature of owner1 is incorrect (446ms)
  ✓ shouldn't mint saito tokens if signature of owner2 is incorrect (207ms)
  ✓ shouldn't mint saito tokens if signature of owner3 is incorrect (211ms)
  ✓ shouldn't mint saito tokens if  nonce is incorrect (320ms)
  ✓ should increment minting nonce by an owner correct (192ms)
  ✓ shouldn't increment minting nonce by not an owner (100ms)
  ✓ should burn tokens correct (401ms)
  ✓ shouldn't burn too big amount of saito token (340ms)
  ✓ should transfer saito tokens correct (544ms)
  ✓ should approve saito tokens correct (632ms)

17 passing (9s)
```

FILE	% STMTS	% BRANCH	% FUNCS	% LINES	UNCOVERED LINES
contracts\	100.00	100.00	100.00	100.00	
SaitoToken.sol	100.00	100.00	100.00	100.00	
All files	**100.00**	**100.00**	**100.00**	**100.00**	

We are grateful to have been given the opportunity to work with the Saito team.

The statements made in this document should not be interpreted as investment or legal advice, nor should its authors be held accountable for decisions made based on them.

Zokyo's Security Team recommends that the Saito team put in place a bug bounty program to encourage further analysis of the smart contract by third parties.

ZOKYO.